Lab 3 – Conversion from NFA to DFA

Aim

To convert a Non-deterministic Finite Automaton (NFA) into an equivalent Deterministic Finite Automaton (DFA) using the subset construction method.

Algorithm

1. Start the program.
2. Read the number of states in the NFA.
3. Read the transition table for each state.
4. Take the start state of the NFA.
5. Compute the **ε-closure** of the start state (if ε-transitions exist).
   For this simplified lab, direct transitions are used.
6. Treat the resulting set of NFA states as the first DFA state.
7. For each DFA state and each input symbol:
    1. Find all reachable NFA states.
    2. Form a new set.
8. If the set is not already present in the DFA state list, add it as a new DFA state.
9. Continue until no new DFA states are generated.
10. Display the DFA transition table.
11. Stop.

C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 10

int n;
char symbols[2] = {'0', '1'};
char nfa[MAX][2][MAX];
char dfa[MAX][MAX];
int dfaCount = 0;

int exists(char *state) {
   for (int i = 0; i < dfaCount; i++) {
      if (strcmp(dfa[i], state) == 0)
         return 1;
   }
   return 0;
}

void sortString(char *str) {
   int len = strlen(str);
   for (int i = 0; i < len - 1; i++) {
      for (int j = i + 1; j < len; j++) {
```

```
            if (str[i] > str[j]) {
                char temp = str[i];
                str[i] = str[j];
                str[j] = temp;
            }
        }
    }
}

void removeDuplicates(char *str) {
    char temp[MAX];
    int k = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        int found = 0;
        for (int j = 0; j < k; j++) {
            if (temp[j] == str[i]) {
                found = 1;
                break;
            }
        }
        if (!found)
            temp[k++] = str[i];
    }
    temp[k] = '\0';
    strcpy(str, temp);
}

void move(char *state, int symbolIndex, char *result) {
    result[0] = '\0';

    for (int i = 0; state[i] != '\0'; i++) {
        int s = state[i] - '0';
        strcat(result, nfa[s][symbolIndex]);
    }

    sortString(result);
    removeDuplicates(result);
}

int main() {
    char temp[MAX];
    int i, j;

    printf("Enter number of NFA states: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        for (j = 0; j < 2; j++) {
            printf("Transition from state %d on %c: ", i, symbols[j]);
            scanf("%s", nfa[i][j]);
```

```
        }
    }

    strcpy(dfa[dfaCount++], "0");

    printf("\nDFA Transition Table:\n");

    for (i = 0; i < dfaCount; i++) {
        printf("\nState %s\n", dfa[i]);

        for (j = 0; j < 2; j++) {
            move(dfa[i], j, temp);

            if (strlen(temp) == 0)
                strcpy(temp, "-");

            printf("  on %c -> %s\n", symbols[j], temp);

            if (strcmp(temp, "-") != 0 && !exists(temp)) {
                strcpy(dfa[dfaCount++], temp);
            }
        }
    }

    return 0;
}
```

Sample Input

Enter number of NFA states: 3

Transition from state 0 on 0: 01
Transition from state 0 on 1: 0

Transition from state 1 on 0: 2
Transition from state 1 on 1: 2

Transition from state 2 on 0: 2
Transition from state 2 on 1: 2

Sample Output

DFA Transition Table:

State 0
  on 0 -> 01
  on 1 -> 0

State 01
  on 0 -> 012

on 1 -> 02

State 012
on 0 -> 012
on 1 -> 02

State 02
on 0 -> 012
on 1 -> 02

Result

Thus the conversion of NFA to DFA was implemented successfully using the subset construction method.